K8S-02: DynaKube Operator Deployment

> **Series:** K8S — Kubernetes Monitoring | **Notebook:** 2 of 14 | **Created:** January 2026 | **Last Updated:** 10/15/2026

Installing and Configuring the Dynatrace Operator

The DynaKube operator is the recommended way to deploy Dynatrace monitoring in Kubernetes. This notebook covers installation via Helm, configuration options, and deployment modes for different use cases.

Table of Contents

1. Operator Overview
2. Prerequisites Setup
3. Helm Chart Installation
4. DynaKube Custom Resource
5. Deployment Modes Explained
6. Configuration Options
7. Verification and Validation
8. Upgrading the Operator

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS tenant with API tokens
Kubernetes Cluster	v1.24+ with admin access
Helm	v3.x installed
kubectl	Configured for your cluster
API Tokens	Operator token (Platform Token recommended) + Data ingest token

> **Operator token type:** new automation should use **Platform Tokens** (`dt0s16`) with `Authorization: Bearer` for the Operator token — note that **the Operator itself accepts platform tokens only from Operator 1.10.0** (released July 15, 2026): *"No immediate action is required and existing access tokens continue to be accepted."* (Dynatrace Operator 1.10.0 release notes (DT docs)) On clusters running an earlier Operator, keep the classic access (API) token — it remains the working path. The OneAgent installer download path still uses Classic API Tokens (`dt0c01` with `Authorization: Api-Token`); this is expected and supported. See the IAM series for parameterized policy patterns and the ONBRD-99 Recommended Defaults card for the canonical token / configuration / extension stack.
>
> **Version Support Policy:** OneAgent and ActiveGate versions are supported for **9 months (Standard)** or **12 months (Enterprise)** from release. The Operator is not in that table — for Operator versions, follow the upgrade path in each release's notes (Operator 1.11.0: *"Minimum operator version required for direct upgrade: 1.6.0"*). Third-party technologies are supported for 6 months beyond vendor EOL. See Support Policy (Dynatrace).

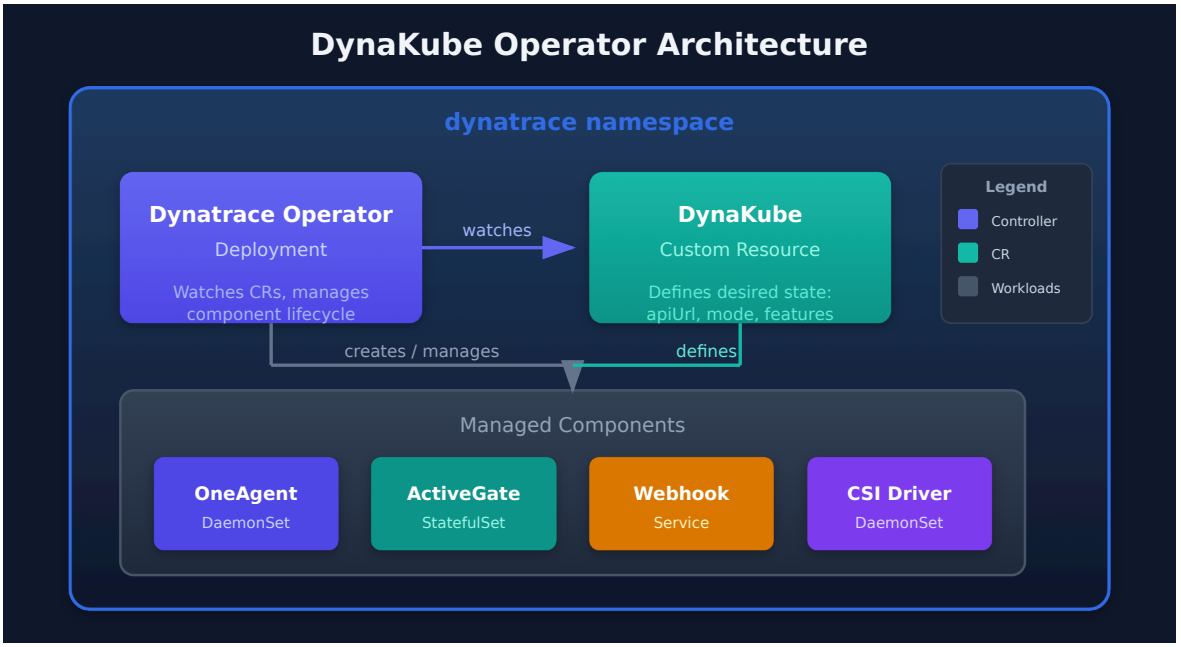
1. Operator Overview

The Dynatrace Operator manages the complete lifecycle of Dynatrace monitoring components.

What the Operator Manages

Component	Description	Managed By
OneAgent DaemonSet	Node-level monitoring	Operator
ActiveGate StatefulSet	Routing and K8s API access	Operator
Webhook	Code module injection	Operator
CSI Driver	Volume-based code modules, cached per node	Operator — on by default with Helm; absent from the plain `kubernetes.yaml` manifest (`kubernetes-csi.yaml` adds it)
Image volumes	Code modules from a node-shared image volume — no CSI DaemonSet (Operator 1.11.0+, Kubernetes 1.35+)	Kubernetes, configured by the Operator

Operator Architecture



2. Prerequisites Setup

Required API Tokens

> **Platform tokens — available now (Dynatrace Operator 1.10.0, released 07/15/2026):** the Operator accepts a **platform token** in the `apiToken` field as the successor to the classic access token. Platform tokens carry fine-grained, user-permission-based scoping, can be rotated or temporarily disabled without deletion, and support configurable expiry. Supplying one also turns on **public-registry image resolution** — Migrate to public registry (DT docs): *"When your DynaKube uses a platform token, Dynatrace Operator enables public registry resolution automatically. No annotation is required."* Without a platform token, the same feature is opted into with the `feature.dynatrace.com/use-public-registry: "true"` DynaKube annotation. Three points from that page:
>
> - It requires **Dynatrace SaaS 1.343 or later**.
> - *"All managed component pods restart when the feature is first enabled."* — so moving an existing DynaKube to a platform token restarts its managed component pods (ActiveGate, OneAgent). Schedule it as a change window, not as a routine credential rotation.
> - **Not every component resolves on Operator 1.10.** *"Container images for Extension Execution Controller, Dynatrace Log Module and the SQL Extension Executor are supported from Dynatrace SaaS 1.350 and Dynatrace Operator 1.11. In older versions, these components always require a image specified in the respective spec.templates field."* Operator 1.11.0 was released 10/01/2026, but its notes say this *"requires a newer Dynatrace SaaS version, currently planned for 1.350"* — SaaS releases roll out to tenants in stages, so until both versions reach your estate, keep setting those three images in `spec.templates`; ActiveGate, OneAgent and code-module images resolve from Operator 1.10.0. The Operator 1.10.0 release notes (updated 09/16/2026) still list EEC, the Log Module and SQL Extension Executor among the auto-resolved images — the how-to is the more specific page, so follow it and verify on your versions.
>
> Any `image` or `codeModulesImage` field already set on the DynaKube takes precedence over the resolved image — remove them when you want resolution to apply.
>
> **Access tokens remain fully supported**; Dynatrace has committed to lead time before platform tokens become mandatory, so the access-token scopes below stay

> the working path. Separately, `spec.tokens.paasToken` is deprecated in favour of `apiToken` — new deployments should not set it.

Create two tokens. The current documentation describes **platform tokens** assigned to a dedicated service user — the **Operator token** with the *Kubernetes Operator* policy, the **Data Ingest token** with the *Kubernetes Ingest* policy — and the Kubernetes onboarding flow creates both automatically:

Operator token (platform token):

Scope	Purpose
`fleet-management:activegate.connection-info:read`	ActiveGate lifecycle information
`fleet-management:activegate.tokens:create`	ActiveGate authentication tokens
`fleet-management:container-images:read`	Image information for managed components
`fleet-management:oneagent.connection-info:read`	OneAgent lifecycle information
`fleet-management:oneagents:download`	OneAgent lifecycle
`settings:objects:read` / `settings:objects:write`	Kubernetes API monitoring, KSPM and log-monitoring settings

Data Ingest token (platform token):

Scope	Purpose
`openpipeline:logs:ingest`	Ingest logs
`openpipeline:metrics:ingest`	Ingest metrics
`openpipeline:traces:ingest`	Ingest traces
`storage:metrics:write`	Write metrics

Classic access tokens remain accepted (*"existing access tokens continue to be accepted"*), and Operators before 1.10.0 accept only them. If a token is missing a scope, the DynaKube status **Tokens** condition names it — check it with `kubectl -n dynatrace describe dynakube` rather than guessing.

> **Sources:** Tokens and permissions (DT docs) — *"Operator token —assigned the Kubernetes Operator policy. Manages the lifecycle of all Dynatrace components in the cluster."*

Create Namespace and Secret

```
# Create namespace
kubectl create namespace dynatrace

# Create secret with tokens
kubectl create secret generic dynakube \
  --namespace dynatrace \
  --from-literal=apiToken=&lt;OPERATOR_TOKEN&gt; \
  --from-literal=dataIngestToken=&lt;DATA_INGEST_TOKEN&gt;
```

3. Helm Chart Installation

Chart Reference: OCI (documented) or Classic Repository

Dynatrace documents the operator chart as an **OCI reference** on a public registry:

```
# Documented form — OCI registry, version pinned explicitly
helm upgrade dynatrace-operator oci://public.ecr.aws/dynatrace/dynatrace-
operator \
  --version 1.10.2 \
  --create-namespace \
  --namespace dynatrace \
  --install \
  --atomic
```

`--install` makes the same command serve a first install and an upgrade; `--atomic` rolls the release back automatically if any component fails to start.

The classic `helm repo add` index is still served and still works, so automation built on it is **stale, not broken** — there is no urgency to rewrite it, and it is not the long-deprecated `dynatrace/helm-charts` repository. Prefer the OCI form for new automation; the classic form remains useful for `helm search repo` version discovery:

```
# Classic form — still functional
helm repo add dynatrace https://raw.githubusercontent.com/Dynatrace/dynatrace-operator/main/config/helm/repos/stable
helm repo update

# View available versions
helm search repo dynatrace/dynatrace-operator --versions

# Install operator only (DynaKube CR applied separately)
helm install dynatrace-operator dynatrace/dynatrace-operator \
  --namespace dynatrace \
  --create-namespace \
  --wait
```

> **Always pass `--version`.** An unpinned install silently takes whatever is newest at run time, which is how a cluster ends up on a release its own notes tell you to skip. §8 covers how to choose the version.

What the Install Creates

Component	Kind	Role
`dynatrace-operator`	Deployment	Reconciles DynaKube CRs
`dynatrace-webhook`	Deployment	Admission webhook — performs code-module injection
`dynatrace-oneagent-csi-driver`	DaemonSet	Serves OneAgent code modules to pods as volumes (when `csidriver.enabled: true`)
`webhook-cert-generator`	init container	Creates the `dynatrace-webhook-certs` secret the webhook needs before it can start (Operator 1.10.0+)
`crd-storage-migrator`	init container	Migrates stored DynaKube objects to the current CRD storage version (Operator 1.10.0+)

Naming the two init containers matters during triage: an operator pod stuck at `Init:0/1` is waiting on one of them, and

```
kubectl -n dynatrace get pod -l app.kubernetes.io/name=dynatrace-operator \
  -o jsonpath='{.items[*].spec.initContainers[*].name}'
```

tells you which is present. K8S-09 §2 maps each to its failure mode.

Install with Custom Values

```
# Create values file
cat &gt; dt-values.yaml &lt;&lt; 'EOF'
# Operator image — top-level chart values, not under "operator:".
# Leave empty for the chart default; set only for a private mirror.
imageRef:
  repository: ""
  tag: ""
customPullSecret: ""  # Required only when pulling from a private registry

# Platform-specific settings
platform: "kubernetes"  # or "openshift"

# Webhook configuration
webhook:
  hostNetwork: false

# CSI driver
csidriver:
  enabled: true
EOF

# Install with values
helm upgrade dynatrace-operator oci://public.ecr.aws/dynatrace/dynatrace-
operator \
  --version 1.10.2 \
  --namespace dynatrace \
  --create-namespace \
  --values dt-values.yaml \
  --install \
  --atomic
```

> **Private mirror vs. public-registry auto-update (Operator 1.10.0+).** The chart's top-level `imageRef` (or the older `image`, which the chart's values.yaml (Dynatrace GitHub) marks *"OBSOLETE -> use imageref instead!"*) and `customPullSecret` set only the Operator's own image. Component images (ActiveGate, OneAgent, code modules) are resolved from the public registry when the DynaKube uses a **platform**

> **token** or carries the `feature.dynatrace.com/use-public-registry: "true"` annotation, and no `image` / `codeModulesImage` field overrides them (see §2 and Migrate to public registry (DT docs)). That is what enables component **auto-update with multi-architecture image support** — **ARM64, x86-64, s390x, and PPC64le** — so a mixed-architecture node pool resolves the right image per node with no per-arch configuration. Mirroring into a private registry is fully supported, but you then own re-mirroring every architecture you run, and auto-update can only pick up what you have mirrored. Decide this before a mixed-arch rollout, not during one. Operator 1.10.0 was released July 15, 2026 and estates adopt it on their own schedule — on earlier Operators the pre-1.10 pull behavior remains the working path, unchanged.

4. DynaKube Custom Resource

The DynaKube CR defines how monitoring is deployed.

Minimal DynaKube (cloudNativeFullStack)

```
apiVersion: dynatrace.com/v1beta6
kind: DynaKube
metadata:
  name: dynakube
  namespace: dynatrace
spec:
  # Your Dynatrace environment URL
  apiUrl: https://ENVIRONMENT_ID.live.dynatrace.com/api

  # OneAgent deployment mode
  oneAgent:
    cloudNativeFullStack:
      tolerations:
        - effect: NoSchedule
          key: node-role.kubernetes.io/master
          operator: Exists
        - effect: NoSchedule
          key: node-role.kubernetes.io/control-plane
          operator: Exists

  # ActiveGate for Kubernetes monitoring
  activeGate:
    capabilities:
      - kubernetes-monitoring
      - routing
      - dynatrace-api
```

> **API Version Note.** `v1beta6` is the current DynaKube API version (introduced in Operator 1.8.0) and is what the examples in this notebook use — the production CR in K8S-14 is written against it too. `v1beta5` is still *served*, but since Operator 1.10.0 it is flagged `deprecated: true` in the CRD — treat it as the next migration, not a resting place.
>
> The deprecation train is what makes this load-bearing at upgrade time:
>
> Read from the shipped CRDs rather than from prose (verified 08/27/2026 and, for 1.11.0, 10/02/2026, against the `kubernetes.yaml` released with each version):

API version	1.9.0	1.10.0 – 1.10.2	1.11.0	What it means
`v1beta6`	served, **storage**	served, **storage**	served, **storage**	Current — use for new DynaKubes
`v1beta5`	served	served, **deprecated**	served, **deprecated**	Still applies, but on the deprecation train
`v1beta4`	served, deprecated	**NOT served**	**removed from the CRD**	**Breaking at 1.10.0**; gone entirely in 1.11.0
`v1beta3`	absent	absent	absent	Removed in 1.9.0

`v1beta4` is not a "migrate at your convenience" case. It was deprecated in **1.9.0** (not 1.10.x) and
stopped being served in 1.10.0 — so a cluster still applying `v1beta4` manifests breaks on the 1.10.x
upgrade exactly as `v1beta3` broke on 1.9.0. Migrate off it *before* the upgrade, not after.

Audit before crossing a version that removes one. Check what the cluster currently serves:

```
bash &gt; kubectl get dynakube -A -o jsonpath='{.items[*].apiVersion}' &gt;
```

That reports the version the API server serves, which is not necessarily the string in your committed YAML — so also grep the manifests your GitOps controller applies (`grep -r "apiVersion: dynatrace.com/" .`). A manifest pinned to a removed version fails at apply time, which in a GitOps setup surfaces as a sync error rather than an obvious upgrade failure. §8 folds this into the pre-upgrade checklist.

Apply the DynaKube

```
# Apply DynaKube CR
kubectl apply -f dynakube.yaml

# Watch deployment progress
kubectl -n dynatrace get dynakube -w
```

5. Deployment Modes Explained

cloudNativeFullStack (Recommended)

```
oneAgent:
  cloudNativeFullStack:
    # Injected via webhook — CSI driver volumes (Helm default), ephemeral
volumes (no CSI driver) or image volumes (Operator 1.11.0+)
```

Pros	Cons
No privileged containers for apps	CSI driver by default with Helm (ephemeral volumes without it; image volumes from Operator 1.11.0 — see below and K8S-12 §2)
Best for multi-tenant clusters	Slightly more complex
Independent app/infra monitoring	

> **Image volumes (Operator 1.11.0+, released 10/01/2026).** Per the 1.11.0 release notes, image volume-based code-module injection *"replaces the CSI driver as the recommended approach"*: each node pulls the code-modules image once and shares it with every instrumented pod, with no CSI DaemonSet and no per-pod copy. Prerequisites: **Kubernetes 1.35+**, **containerd 2.2+ or CRI-O 1.33+**, and a code-modules image from the public registry or a private mirror of it — *"Image volume injection is not compatible with the Dynatrace built-in tenant registry."* Enable it with the DynaKube annotation `feature.dynatrace.com/mount-code-modules-via-image-volume: "true"` (mutually exclusive with `feature.dynatrace.com/node-image-pull`), test first on single pods with `oneagent.dynatrace.com/volume-type: "image"`, and plan a rolling restart: *"A full migration requires a rolling restart of all injected workloads."* On older clusters or Operators, the CSI driver or ephemeral volumes remain the working path.
>
> **Sources:** Operator 1.11.0 release notes (DT docs), Use image volumes for code modules injection (DT docs), Migrate to image volumes (DT docs).

classicFullStack (Legacy — Migrate Existing Clusters, Not Just New Ones)

```
oneAgent:
  classicFullStack:
    # OneAgent DaemonSet provides code modules via host mounts
```

Pros	Cons
Simpler architecture	All pods share OneAgent
Single component	Requires hostPath mounts
	Not a ready state for the latest Dynatrace — see below

> **"Avoid for new deployments" understates this.** The ready-made *Check your upgrade readiness* dashboard treats a DynaKube using `classicFullStack` as **not ready** for the latest Dynatrace — the check is on the cluster's current state, not on when it was created. A cluster that has been running `classicFullStack` happily for two years is flagged exactly like a new one.
>
> So this is migration work with a deadline attached, not a style preference. Move existing clusters to `cloudNativeFullStack` (or `applicationMonitoring` / `hostMonitoring` where those fit) as part of upgrade preparation rather than waiting for a rebuild. The readiness check also covers Operator ≥ 1.10, the connecting ActiveGate ≥ 1.343, and the image source — see §3 and the Classic → Gen3 doorway in the `-START-HERE-` playbook.

applicationMonitoring

```
oneAgent:
  applicationMonitoring: {}
  # Only application-level monitoring, no infrastructure.
  # Whether the CSI driver is used is set when installing the Operator (§3),
not here.
```

> **`useCSIDriver` is not a field on current DynaKube API versions.** The parameters reference lists it under `applicationMonitoring` only for `v1beta2` and `v1beta1`, and

> `v1beta2` is gone from Operator 1.7.0 onward. On `v1beta3` and later, whether code modules come from the CSI driver is decided when the Operator is installed: the install guide has a CSI variant and a *Without CSI driver* variant, and the Helm values in §3 carry a `csidriver` block.
>
> **Sources:** DynaKube parameters (DT docs) — *"DynaKube API version v1beta2 is no longer available with Dynatrace Operator version 1.7.0"*; Application observability setup (DT docs) — *"CSI driver is optional (see step 2). If enabled, it gets deployed as DaemonSet and results in a CSI driver pod on each node."*

Pros	Cons
Minimal footprint	No infrastructure visibility
No node-level access needed	No container metrics
Good for managed K8s	

hostMonitoring

```
oneAgent:
  hostMonitoring: {}
```

Pros	Cons
Infrastructure only	No application tracing
Lightweight	No code-level insight

6. Configuration Options

Resource Limits

```
spec:
  oneAgent:
    cloudNativeFullStack:
      oneAgentResources:      # resources of the OneAgent DaemonSet pods
        requests:
          cpu: 100m
          memory: 256Mi
        limits:
          cpu: 300m
          memory: 512Mi
```

Node Selectors and Tolerations

```
spec:
  oneAgent:
    cloudNativeFullStack:
      nodeSelector:
        kubernetes.io/os: linux
      tolerations:
        - key: "dedicated"
          operator: "Equal"
          value: "monitoring"
          effect: "NoSchedule"
```

Namespace Selectors

Control which namespaces get injection. The selector belongs to the **injection mode**, not to `spec` — a top-level `spec.namespaceSelector` is not a DynaKube field and is ignored:

```
spec:
  oneAgent:
    cloudNativeFullStack:
      namespaceSelector:
        matchLabels:
          dynatrace-injection: enabled
```

Or exclude specific namespaces:

```
spec:
  oneAgent:
    cloudNativeFullStack:
      namespaceSelector:
        matchExpressions:
          - key: dynatrace-injection
            operator: NotIn
            values:
              - disabled
```

`applicationMonitoring` takes the same `namespaceSelector`, and `spec.metadataEnrichment.namespaceSelector` scopes metadata enrichment separately. Field paths checked against the `v1beta6` schema in the Operator 1.11.0 CRD, 10/02/2026.

Custom ActiveGate Configuration

```
spec:
  activeGate:
    capabilities:
      - kubernetes-monitoring
      - routing
    replicas: 2
    resources:
      requests:
        cpu: 500m
        memory: 512Mi
      limits:
        cpu: 1000m
        memory: 1Gi
    topologySpreadConstraints:
      - maxSkew: 1
        topologyKey: topology.kubernetes.io/zone
        whenUnsatisfiable: ScheduleAnyway
```

Feature Flags and Annotations

Control injection at the pod level:

Annotation	Effect
`oneagent.dynatrace.com/inject: "false"`	Disable injection for pod

Annotation	Effect
`oneagent.dynatrace.com/inject: "true"`	No effect — the inject annotations only **exclude** (*"setting it to true will have no effect"*); inclusion comes from the namespace selector
`oneagent.dynatrace.com/technologies: "java,nodejs"`	Limit the code modules downloaded — *"Ignored if the CSI volume is used or node image pull via ephemeral volume is used"*

> **Sources:** Annotate pods (DT docs).

DynaKube feature flags:

```
spec:
  oneAgent:
    cloudNativeFullStack:
      # Note: ONEAGENT_ENABLE_VOLUME_STORAGE defaults to true when CSI driver
      # is enabled — no need to set explicitly
```

Templates Configuration

Configure OTel Collector and Log Monitoring via `spec.templates`:

```
spec:
  templates:
    otelCollector:
      # ⚠️ Always pin to specific version - avoid 'latest'.
      # Example pin (0.56.0, Sept 2026) — check the releases before
installing:
      # https://github.com/Dynatrace/dynatrace-otel-collector/releases
      imageRef:
        repository: public.ecr.aws/dynatrace/dynatrace-otel-collector
        tag: "0.56.0"
      resources:
        requests:
          cpu: 100m
          memory: 256Mi
        limits:
          cpu: 500m
          memory: 512Mi

    logMonitoring:
      resources:
        requests:
          cpu: 100m
          memory: 256Mi
        limits:
          cpu: 500m
          memory: 512Mi
      tolerations:
        - effect: NoSchedule
          key: node-role.kubernetes.io/control-plane
          operator: Exists
```

> **Note:** Log monitoring is enabled with `spec.logMonitoring: {}` but resource configuration goes under `spec.templates.logMonitoring`.

OTel Collector and Log Monitoring Sizing Guide

Environment	CPU Limit	Memory Limit	Notes
Dev/Test	200m	256Mi	Low telemetry volume
Staging	500m	512Mi	Medium volume
Production	1000m	1Gi	High volume

7. Verification and Validation

Check Operator Status

```
# Operator pods
kubectl -n dynatrace get pods -l app.kubernetes.io/name=dynatrace-operator

# DynaKube status
kubectl -n dynatrace get dynakube

# Detailed status
kubectl -n dynatrace describe dynakube dynakube
```

Check OneAgent Deployment

```
# OneAgent DaemonSet
kubectl -n dynatrace get daemonset -l app.kubernetes.io/component=oneagent

# OneAgent pods on each node
kubectl -n dynatrace get pods -l app.kubernetes.io/component=oneagent -o wide
```

Check ActiveGate

```
# ActiveGate StatefulSet
kubectl -n dynatrace get statefulset -l app.kubernetes.io/component=activegate

# ActiveGate pods
kubectl -n dynatrace get pods -l app.kubernetes.io/component=activegate
```

DynaKube Status Phases

Phase	Meaning
`Running`	All components healthy
`Deploying`	Components being created
`Error`	Check events for details

```
// Verify Kubernetes cluster is reporting to Dynatrace
fetch dt.entity.kubernetes_cluster
| fields entity.name, tags
| sort entity.name asc

// Smartscape equivalent (dt.entity.* is deprecated but still functional):
//   smartscapeNodes "K8S_CLUSTER"
//   | fields name, tags
//   | sort name asc
// Caveat: Smartscape reflects CURRENT live topology and can report fewer
entities than
// the classic entity store; for a pre-migration inventory keep the classic
query above.
// Note: on Smartscape, `tags` is a record of tag key -&gt; value (dictionary:
"Note that rule-based
// tags do not exist in the new model."). Classic rule-based tags such as
dt.owner:... do not appear
// there, so the legacy query and the Smartscape query above do not return the
same tags.
```

```
// Check Dynatrace component health via Kubernetes events in the dynatrace
namespace
// (Kubernetes events are events, not logs — see K8S-01 §6). BackOff /
Unhealthy / Killing
// here point at OneAgent, ActiveGate, webhook or CSI pods that are not
settling.
fetch events, from:-1h
| filter event.provider == "KUBERNETES_EVENT" and k8s.namespace.name ==
"dynatrace"
| fields timestamp, k8s.pod.name, dt.kubernetes.event.reason,
dt.kubernetes.event.message
| sort timestamp desc
| limit 20
```

8. Upgrading the Operator

Pre-Upgrade Checks

An operator upgrade is not only a version bump — it can change **chart defaults** and it can **remove a DynaKube API version**. Both are silent in a values diff. Run these three checks before every upgrade.

1. Will a changed chart default alter my deployment?

`--reuse-values` preserves *the values you set*. It does **not** shield you from a changed default for a value you never set, because there is nothing of yours to reuse. Render the incoming chart and diff it against what is live:

```
# Render the target version with your current values
helm get values dynatrace-operator -n dynatrace &gt; current-values.yaml

helm template dynatrace-operator oci://public.ecr.aws/dynatrace/dynatrace-
operator \
  --version 1.10.2 \
  --namespace dynatrace \
  --values current-values.yaml &gt; new.yaml

# Compare against the rendered manifests currently deployed
helm get manifest dynatrace-operator -n dynatrace &gt; live.yaml
diff live.yaml new.yaml
```

The concrete case this catches: **Operator 1.10.0 flipped the ActiveGate `TopologySpreadConstraint` default from `DoNotSchedule` to `ScheduleAnyway`.** Expect an **ActiveGate restart on upgrade** — the pod spec changes, so the StatefulSet rolls. Nothing is broken; plan for the brief gap in ActiveGate-served capabilities (Kubernetes API monitoring, routing, metrics ingest) rather than being surprised by it.

If your DynaKube sets `whenUnsatisfiable` explicitly, **your value wins** and the default flip does not apply to you. The ActiveGate example in §6 already sets `ScheduleAnyway` explicitly, which is the recommended posture: an explicit constraint makes the behavior independent of chart defaults across upgrades.

2. Does my DynaKube declare an API version the target Operator still serves?

```
# What the cluster serves today
kubectl get dynakube -A -o jsonpath='{.items[*].apiVersion}'

# What your committed manifests declare
grep -r "apiVersion: dynatrace.com/" .
```

`v1beta3` was **removed in Operator 1.9.0**, `v1beta4` **stopped being served in 1.10.0** and was **removed from the CRD in 1.11.0** — against either, an apply fails outright once you cross that version. The 1.11.0 notes add that *"depending on your current version, upgrading directly to 1.11 may not be possible until the migration is done"*. `v1beta4` was deprecated back in 1.9.0, so a cluster that took the deprecation as advisory is the one that breaks here. `v1beta5` is still served but is flagged deprecated from 1.10.0. Target `v1beta6` for anything you are rewriting anyway. Full table in §4.

3. Will the new injected pod spec still be admitted?

On OpenShift, and on any cluster with restrictive admission policy, an operator upgrade changes injected pod specs — and those specs are re-validated on admission. **FAQ-13 §6 ("Assessing Impact Before You Upgrade")** generalizes this into a pre-upgrade assessment checklist; run it before an OpenShift upgrade rather than after.

Helm Upgrade Process

```
# Check current version
helm list -n dynatrace

# Upgrade to a specific, validated version (OCI — documented form)
helm upgrade dynatrace-operator oci://public.ecr.aws/dynatrace/dynatrace-
operator \
  --version 1.10.2 \
  --namespace dynatrace \
  --reuse-values \
  --atomic \
  --wait

# Classic-repo equivalent (still functional)
helm repo update
helm upgrade dynatrace-operator dynatrace/dynatrace-operator \
  --version 1.10.2 \
  --namespace dynatrace \
  --reuse-values \
  --wait
```

> **Pin the target, do not upgrade to "latest".** Omitting `--version` takes whatever is newest at run time — which is how a cluster lands on a release its own notes advise skipping.

Version Compatibility

Dynatrace publishes Kubernetes compatibility per **Kubernetes version**, not per Operator line. Dynatrace Operator support and known issues (DT docs) says *"The table below lists the verified and tested release versions"*: for each Kubernetes and OpenShift version it gives a minimum and a recommended Dynatrace Operator version, and no maximum. Look up your Kubernetes version there before choosing an Operator release.

Operator Version	Notes
1.11.x	**Newest line: v1.11.0** (released 10/01/2026). Removes `v1beta4` from the CRD; adds image-volume injection, `spec.kubernetesMonitoring` (size the two ActiveGate StatefulSets inside one DynaKube), and workload/pod labels and annotations as enrichment sources. Minimum version for a direct upgrade: 1.6.0. Review the release notes for every minor version you cross before adopting it.
1.10.x	**Validated pin in this series: v1.10.2** (July 30, 2026) — fixes a workload/namespace **tagging-precedence regression** (now only the first matching rule per key applies — a behaviour change, see K8S-10), `dynatrace-webhook` `CrashLoopBackOff` on gVisor nodes, and injected pods hanging on the OneAgent-binary download (timeout raised to 15 min); it also logs metadata-enrichment rules it cannot apply instead of dropping them silently. **v1.10.1 remains a working pin** until you upgrade — estates adopt on their own schedule — subject to the OpenShift-manifest caveat in K8S-09 §2. **Skip 1.10.0** — its own release notes advise skipping it (auto-update defect) and it is flagged `prerelease: true` on the GitHub releases page. Release notes per tag: Operator releases (Dynatrace GitHub)

> **Operator support window:** the trust-center support table covers OneAgents and ActiveGates (9 / 12 months), not the Operator. For the Operator, the release notes state the upgrade floor — *"Minimum operator version required for direct upgrade: 1.6.0"* in 1.11.0 — so a cluster below 1.6.0 has to upgrade in steps.

Rollback if Needed

```
# View history
helm history dynatrace-operator -n dynatrace

# Rollback to previous version
helm rollback dynatrace-operator 1 -n dynatrace
```

Next Steps

Now that the operator is deployed, proceed to:

Next Notebook	Topic
K8S-03: GitOps for DynaKube	Manage DynaKube with ArgoCD/Flux
K8S-04: Cluster Health Monitoring	Deep-dive into cluster metrics
K8S-05: Workload Monitoring	Application-level observability

Summary

In this notebook, you learned:

- Dynatrace Operator architecture and components
- Prerequisites: API tokens and namespace setup
- Helm chart installation with custom values
- DynaKube CR structure and configuration
- Deployment modes and when to use each
- Configuration options for resources, selectors, and features
- Verification commands and status phases
- Upgrade and rollback procedures

References

- Setup on Kubernetes — deployment (DT docs) — application-observability, full-stack-observability, platform-observability deployment paths
- DynaKube parameters reference (DT docs) — full CRD spec (renamed from `dynakube-crd`)
- DynaKube feature flags (DT docs) — reference list of supported feature-flag annotations
- Helm chart values.yaml (Dynatrace GitHub) — every Helm install option
- Dynatrace Operator releases (Dynatrace GitHub) — recommended pin is **v1.10.2** (July 30, 2026); v1.10.1 remains a working pin until you upgrade. Skip 1.10.0 (auto-update defect; flagged `prerelease: true`). Check before each install

- Operator 1.10.2 release notes (DT docs)
- Operator 1.11.0 release notes (DT docs) — *"Removed DynaKube API version v1beta4"*; image volumes; `spec.kubernetesMonitoring`
- Dynatrace Operator support and known issues (DT docs)
- Tokens and permissions (DT docs)
- Support policy (Dynatrace) — 9 / 12 months for OneAgents & ActiveGates
- Migrate to public registry (DT docs) — prerequisites include SaaS 1.343+; component pods restart when first enabled

---

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.